



gRPC Streaming

Cliente / Servidor

72.42 Programación de Objetos Distribuidos



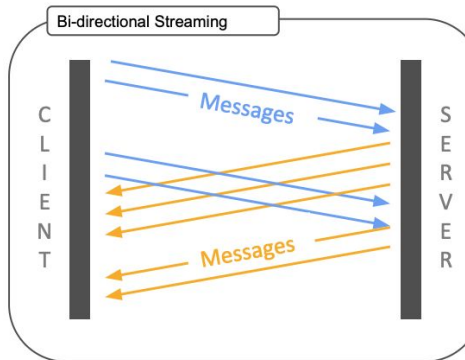
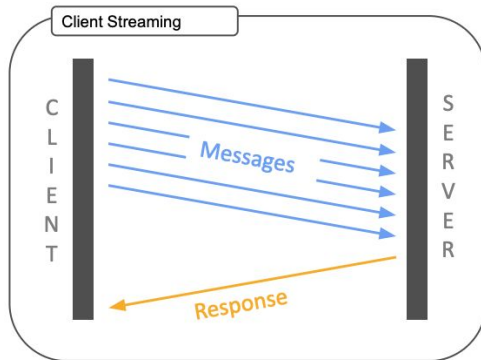
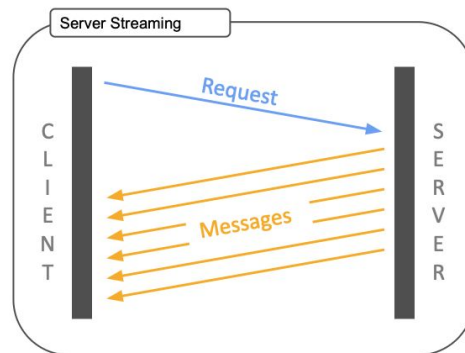
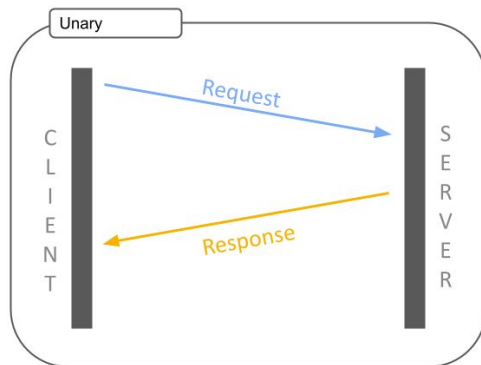
Streaming

gRPC



Tipos de métodos de servicios

- gRPC permite definir **4 tipos de métodos de servicios** de acuerdo al patrón de comunicación.
- Básicamente se toman en cuenta cuántos elementos se mandan del objeto *request* y se reciben del objeto *response*.



RPC Unario

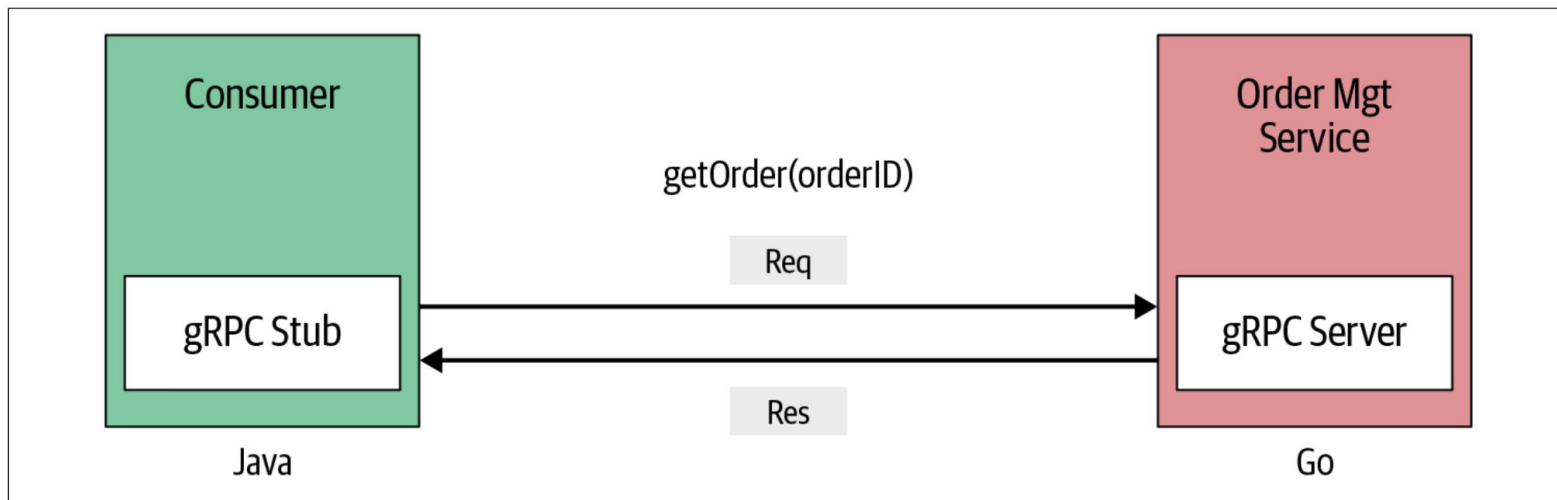
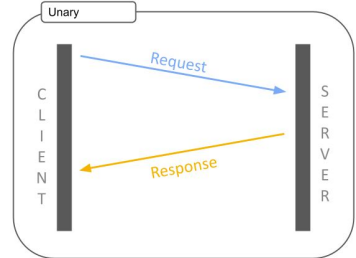


Figure 3-1. Simple/unary RPC

RPC Unario



En RPC Unario el cliente envía un **único objeto *request*** y el servidor responde con un **único objeto *response***.

Todos los ejemplos vistos hasta el momento utilizaron este patrón de comunicación.

```
service TrainTicketService {  
  rpc GetDestinations(google.protobuf.Empty) returns (Destinations);  
}  
  
message Destinations {  
  repeated string destinations = 1;  
}
```

Ejercicio 1: gRPC Tickets - Unario

- **Clonar el repo usando el link de GitHub Classroom (Campus)**
- Abrirlo con un IDE
- Leer el **README.md**
- Implementar el método `GetDestinations` de `TrainTicketService` usando el método `getDestinations` del `TicketRepository`
- Realizar una prueba completa

Streaming

gRPC *streaming* es una variante de comunicación donde *request*, *response* o ambos **envían más de un elemento** del tipo definido.

Para indicar esta repetición se utiliza la palabra reservada **stream** en el elemento que será enviado más de una vez.

```
service TrainTicketService {  
    rpc GetTrainsForDestination(google.protobuf.StringValue) returns (stream Train);  
    rpc PurchaseTicket(stream Ticket) returns (Reservation);  
    rpc GetTicketsForReservations(stream Reservation) returns (stream Ticket);  
}
```

Server Streaming

Client Streaming

Bi-directional Streaming

Streaming vs Repeated

Stream y **repeated** son dos maneras de enviar más de un elemento en la comunicación.

Se pueden intercambiar pero no son lo mismo:

- **stream:** es un modificador para un mensaje (de entrada y/o salida)
- **repeated:** es un modificador para un campo dentro de un mensaje

```
service TrainTicketService {  
    rpc GetDestinations(google.protobuf.Empty) returns (Destinations);  
    rpc GetTrainsForDestination(google.protobuf.StringValue) returns (stream Train);  
}  
message Destinations {  
    repeated string destinations = 1;  
}
```

Técnicamente **repeated** se usa para **listas** que se obtienen **inmediatamente** y **stream** se usa más cuando se tarda en recuperar los elementos.



Streaming

- Internamente la posibilidad de establecer una comunicación vía streaming se basa en las características de HTTP/2 de poder establecer **una conexión de larga duración** y multiplexar los requests/responses.
 - De esta manera cliente y servidor permanecen conectados por un largo período y a medida que se calculan los valores a enviar o retornar se van enviando por el cable.
- 

Server Side Streaming

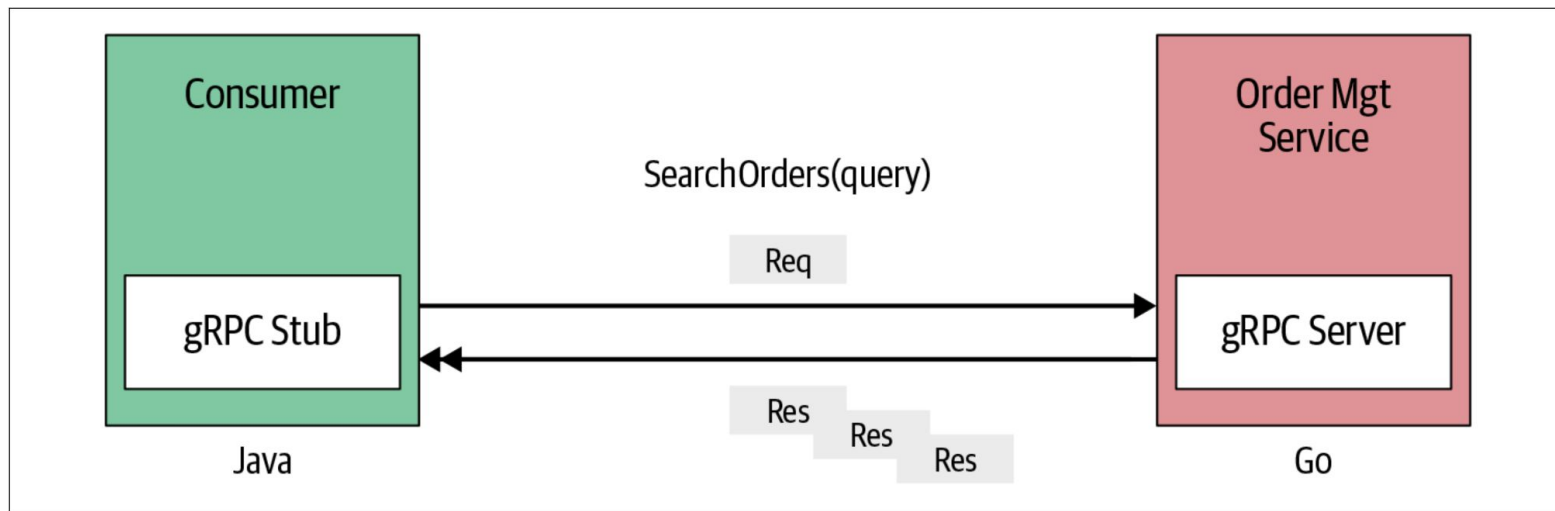
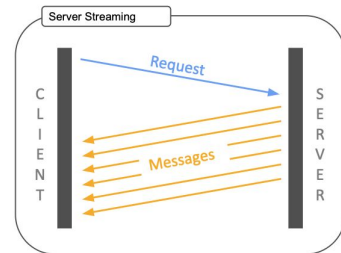


Figure 3-2. Server-streaming RPC

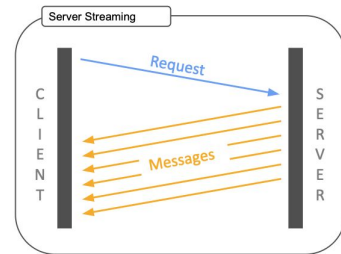
Server Side Streaming



En Server Side Streaming el cliente envía un único elemento y **el servidor retorna más de un elemento.**

```
service TrainTicketService {  
  rpc GetTrainsForDestination(google.protobuf.StringValue) returns (stream Train);  
}  
  
message Train {  
  string id = 1;  
  string destination = 2;  
  string time = 3;  
  int32 available_count = 4;  
}
```

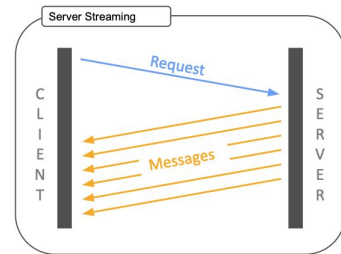
Server Side Streaming



En el Servant la implementación es similar a la del unario pero aquí se invoca más de una vez al **onNext**

```
@Override
public void getTrainsForDestination(StringValue request, StreamObserver<Train>
responseObserver) {
    var destination = request.getValue();
    var availability = ticketRepository.getAvailability(destination);
    availability.forEach(localTrain -> {
        var trainResponse = Train.newBuilder()
            .setId(localTrain.id())
            .setDestination(localTrain.destination())
            .setTime(localTrain.time())
            .setAvailableSeats(localTrain.availableSeats())
            .build();
        responseObserver.onNext(trainResponse);
    });
    responseObserver.onCompleted();
}
```

Server Side Streaming



El Cliente recibe un **iterador** de respuestas.

La implementación `ClientClass#BlockingResponseStream` realiza un `while(true)` en el `hasNext` para **bloquearse** hasta recibir una respuesta.

```
StringValue request = StringValue.of("Mar del Plata");

Iterator<Train> trainsForDestination = blockingStub.getTrainsForDestination(request);

while (trainsForDestination.hasNext()) {
    IO.println(trainsForDestination.next());
}
```

En este caso también se puede utilizar el `*stub` y pasarle un `StreamObserver`.

Ejercicio 2: gRPC Tickets - Server Side Streaming

- Implementar el método `GetTrainsForDestination` usando el método `getAvailability` del `TicketRepository`
- Realizar una prueba completa

Client Side Streaming

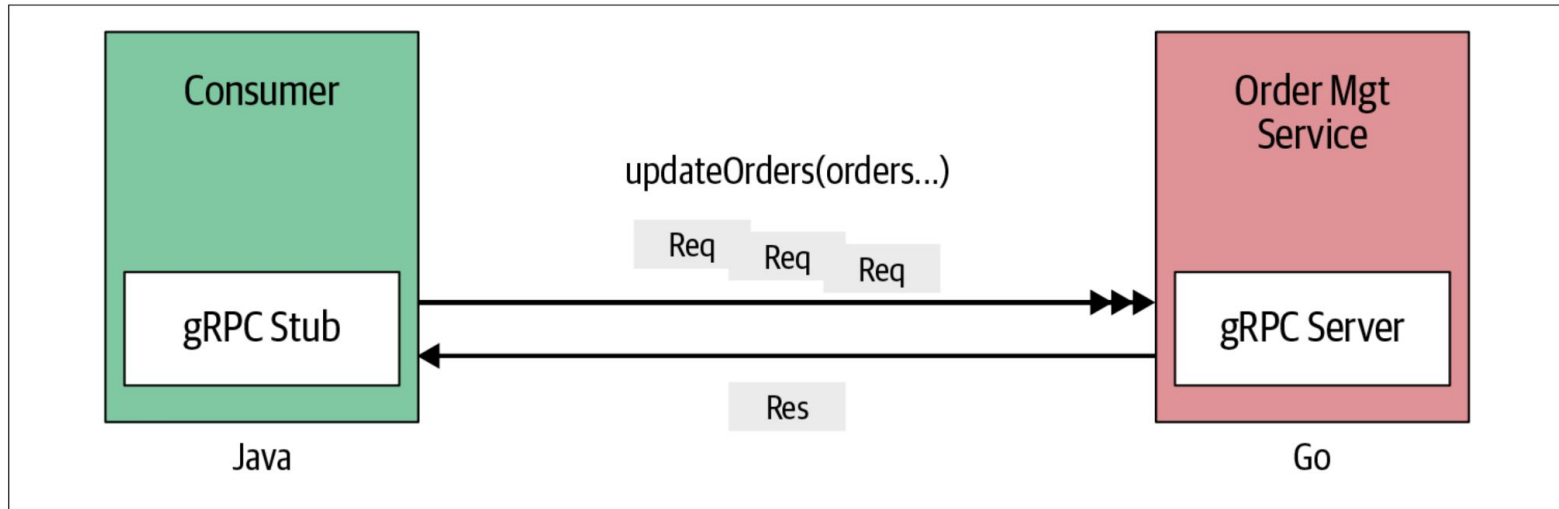
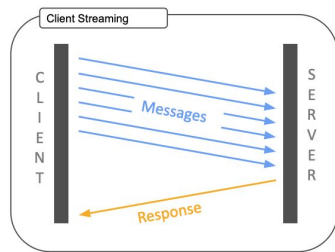


Figure 3-3. Client-streaming RPC

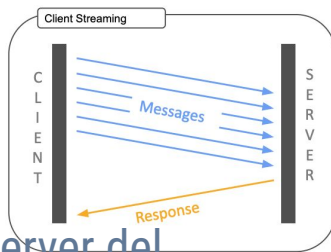
Client Side Streaming



En Client Side Streaming **el cliente envía más de un elemento** y el servidor retorna un único elemento.

```
service TrainTicketService {  
  rpc PurchaseTicket(stream Ticket) returns (Reservation);  
}  
message Ticket {  
  string id = 1;  
  string trainId = 2;  
  string passenger_name = 3;  
}  
message Reservation {  
  string id = 1;  
  int32 ticket_count = 2;  
}
```

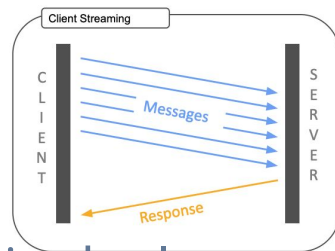

Client Side Streaming



En el Servant, además de recibir el observer para la respuesta, retorna un observer del request para ir aplicando las operaciones “a medida” que el cliente envía un elemento.

```
@Override
public StreamObserver<Ticket> purchaseTicket(StreamObserver<Reservation> responseObserver) {
    return new StreamObserver<>() {
        private final SequencedCollection<Ticket> tickets = new ArrayList<>();
        @Override public void onNext(Ticket ticket) { tickets.add(ticket); }
        @Override public void onError(Throwable throwable) { ... }
        @Override public void onCompleted() {
            var reservationId = ticketRepository.addReservation(tickets);
            var reservation = Reservation.newBuilder()
                .setId(reservationId).setTicketCount(tickets.size()).build();
            responseObserver.onNext(reservation);
            responseObserver.onCompleted();
        }
    };
}
```

Client Side Streaming



El cliente define el observer para la respuesta y llama al método recibiendo el observer del request que se usa para enviar los elementos

```
var responseFuture = new CompletableFuture<Reservation>();
var responseObserver = new StreamObserver<Reservation>() {
    @Override public void onNext(Reservation r) { responseFuture.complete(r); }
    @Override public void onError(Throwable throwable) { ... }
    @Override public void onCompleted() { .. }
};

StreamObserver<Ticket> requestObserver = stub.purchaseTicket(responseObserver);
List.of("John", "Paul", "Ringo").forEach(name -> {
    Ticket ticket = Ticket.newBuilder()
        .setId(UUID.randomUUID().toString())
        .setPassengerName(name).setTrainId(train).build();
    requestObserver.onNext(ticket);
});

requestObserver.onCompleted();
Reservation reservation = responseFuture.get();
```

En este caso el único cliente que se puede utilizar es el *stub.

Ejercicio 3: gRPC Tickets - Client Side Streaming

- Implementar el método `PurchaseTicket` usando el método `addReservation` del `TicketRepository`
- Realizar una prueba completa

Bidirectional Streaming

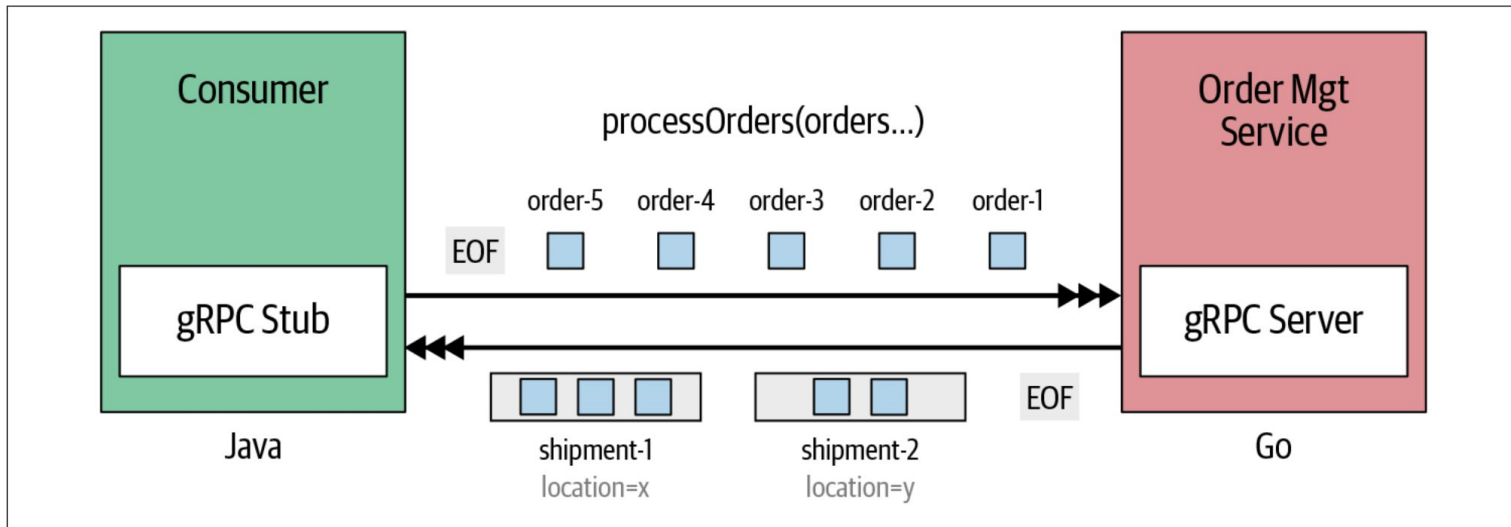
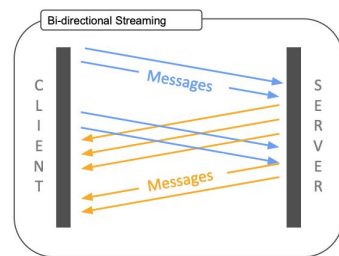


Figure 3-4. Bidirectional-streaming RPC

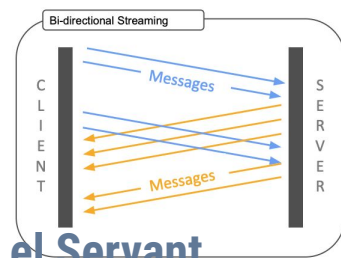
Bidirectional Streaming



En Bidirectional Streaming tanto **cliente** y **servidor** envían más de un valor entre ellos.

```
service TrainTicketService {  
  rpc GetTicketsForReservations(stream Reservation) returns (stream Ticket);  
}
```

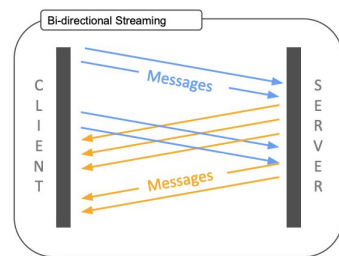
Bidirectional Streaming



En el Servant el código es igual que en Client Streaming, la diferencia es que **el Servant puede enviar más de una respuesta por cada elemento** en vez de esperar a recibir todos los elementos del cliente.

```
@Override
public StreamObserver<Reservation> getTicketsForReservations(StreamObserver<Ticket> responseObserver) {
    return new StreamObserver<>() {
        @Override public void onNext(Reservation reservation) {
            var reservationId = reservation.getId();
            Optional<SequencedCollection<Ticket>> tickets = ticketRepo.getReservation(reservationId);
            if (tickets.isEmpty()) {
                responseObserver.onError(...);
                return;
            }
            tickets.get().forEach(responseObserver::onNext);
        }
        @Override public void onError(Throwable throwable) { ... }
        @Override public void onCompleted() { responseObserver.onCompleted(); }
    };
}
```

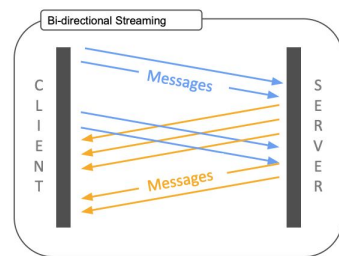
Bidirectional Streaming



El cliente define un observer para las respuestas y llama recibiendo el observer para enviar los requests.

```
SequencedCollection<Ticket> tickets = new ArrayList<>();
var responseLatch = new CountDownLatch(1);
var responseObserver = new StreamObserver<Ticket>() {
    @Override
    public void onNext(Ticket ticket) {
        tickets.add(ticket);
    }
    @Override public void onError(Throwable throwable) { ... }
    @Override
    public void onCompleted() {
        responseLatch.countDown();
    }
};
```

Bidirectional Streaming



```
StreamObserver<Reservation> requestObserver =  
    stub.getTicketsForReservations(responseObserver);  
  
reservationIds.forEach(id → {  
    Reservation reservation = Reservation.newBuilder()  
        .setId(id)  
        .build();  
    requestObserver.onNext(reservation);  
});  
requestObserver.onCompleted();  
  
responseLatch.await();  
  
Log.info("Tickets: {}", tickets);
```

En este caso el único cliente que se puede utilizar es el `*stub`.

Ejercicio 4: gRPC Tickets - Bidirectional Streaming

- Implementar el método `GetTicketsForReservations` usando el método `getReservation` del `TicketRepository`
- Realizar una prueba completa



Error Handling

gRPC





Errors

Todo mensaje gRPC puede retornar un valor o un error.

Para errores el framework define mensajes de Status que cada lenguaje implementa.

En principio hay dos modelos de status para utilizar:

- **io.grpc.Status**: el modelo básico del framework.
- **com.google.rpc.Status**: un modelo más rico

Y para enviar los mensajes de manera sincrónica se utiliza `StreamObserver::OnError`, a menos que se esté transmitiendo en Streaming



io.grpc.Status

El error se puede crear indicando uno de los 16 valores posibles de Status y el cliente obtendrá una `StatusRuntimeException` con esa información.

```
...  
responseObserver.onError(Status.ALREADY_EXISTS  
    .withDescription("User already exists")  
    .asRuntimeException());  
...
```

Exception in thread "main" io.grpc.StatusRuntimeException:
ALREADY_EXISTS: User already exists

Si en lugar de invocar a `onError` se obtiene una excepción en el Servant, al cliente sólo le llegará una `StatusRuntimeException` con el código UNKNOWN.

```
...  
throw new NoSuchElementException("This message does not go to the client");  
...
```

Exception in thread "main" io.grpc.StatusRuntimeException: UNKNOWN

com.google.rpc.Status

Con el modelo enriquecido el comportamiento es similar pero se pueden agregar elementos definidos en [error_details.proto](#) para enviar más información

```
ErrorInfo errorInfo = ErrorInfo.newBuilder()  
    .setReason("Resource not found")  
    .setDomain("Product")  
    .build();  
  
com.google.rpc.Status status = com.google.rpc.Status.newBuilder()  
    .setCode(com.google.rpc.Code.NOT_FOUND.getNumber())  
    .setMessage("Product id not found")  
    .addDetails(Any.pack(errorInfo))  
    .build();  
  
responseObserver.onError(  
    io.grpc.protobuf.StatusProto.toStatusRuntimeException(status)  
);
```

com.google.rpc.Status

```
try {  
    // ...  
} catch (StatusRuntimeException ex) {  
    Status status = StatusProto.fromThrowable(ex);  
    if (status != null) {  
        logger.error("Message: {}", status.getMessage());  
        for (Any detail : status.getDetailsList()) {  
            if (detail.is(ErrorInfo.class)) {  
                ErrorInfo errorInfo = detail.unpack(ErrorInfo.class);  
                logger.error("Reason: {}", errorInfo.getReason());  
                logger.error("Domain: {}", errorInfo.getDomain());  
            }  
        }  
    } else {  
        ...  
    }  
}
```

```
2025-04-16 11:49:53 ERROR Client:49 - Message: Product id not found  
2025-04-16 11:49:53 ERROR Client:53 - Reason: Resource not found  
2025-04-16 11:49:53 ERROR Client:54 - Domain: Product
```



Errores en Streams

Con streams hay que tener cuidado porque al llamar al **onError** se va a cortar la comunicación del cliente con el servidor.

En caso de querer mantener la conexión abierta pero ir enviando el error que tal vez uno de los mensajes provocó, se recomienda incorporar el error como parte de la respuesta del servicio.





Interceptors

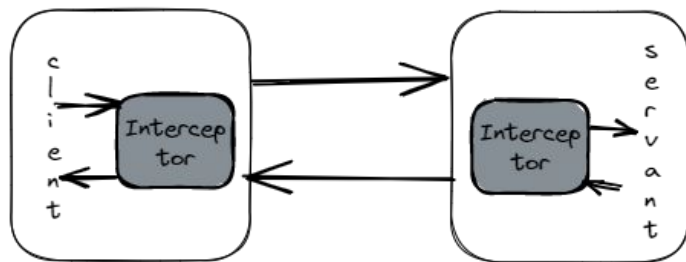
Spring gRPC



Interceptors

Los *interceptors* son elementos opcionales que se pueden definir tanto en el cliente como en el servidor.

La intención es interceptar cada llamado y agregar funcionalidad *cross* a todos los llamados.





Interceptors

Algunos de los casos de uso son:

- Métricas
- Log de los llamados
- Autenticación
- Autorización
- Control de errores
- Agregar Headers especiales o Metadata de contexto.



Los interceptores **pueden encadenarse** por lo que se puede agregar uno por cada responsabilidad

Client Interceptor

Para interceptar el mensaje en el cliente se implementa `ClientInterceptor` y se procesa antes de que se ejecute el llamado (`channel.newCall`)

```
public class ClientLoggerInterceptor implements ClientInterceptor {  
    private static final Logger log = ...  
    @Override  
    public <ReqT, RespT> ClientCall<ReqT, RespT> interceptCall(  
        MethodDescriptor<ReqT, RespT> methodDescriptor,  
        CallOptions callOptions, Channel channel) {  
        // Here goes your code #1  
        String rpcId = UUID.randomUUID().toString();  
        String serviceName = methodDescriptor.getServiceName();  
        String methodName = methodDescriptor.getBareMethodName();  
        log.info("Call {}: {}#{}", rpcId, serviceName, methodName);  
        // Here goes your code #2  
        return channel.newCall(methodDescriptor, callOptions);  
    }  
}
```

```
Call b98861b6-6273-479d-8805-0c88817b0bd3:  
trainTicketService.TrainTicketService#GetDestinations
```

Client Interceptor

Para interceptar la respuesta en el cliente se puede utilizar un `SimpleForwardingClientCall` y overridear el `start`.

```
public class ClientResponseLogger implements ClientInterceptor {
    private static final Logger log = ...
    public <ReqT, RespT> ClientCall<ReqT, RespT> interceptCall(MethodDescriptor<ReqT, RespT> methodDescriptor,
                                                              CallOptions callOptions, Channel channel) {
        return new ForwardingClientCall.SimpleForwardingClientCall<>(
            channel.newCall(methodDescriptor, callOptions)) {
            @Override public void start(Listener<RespT> respListener, Metadata headers) {
                super.start(new ForwardingClientCallListener.SimpleForwardingClientCallListener<>(respListener) {
                    @Override public void onMessage(RespT message) {
                        log.info("Received response from Server: {}", message);
                        super.onMessage(message);
                    }
                }, headers);
            }
        };
    }
}
```

Client Interceptor

En Spring gRPC para agregar un *interceptor* al cliente se define el `@Bean` correspondiente como `@GlobalClientInterceptor` (y si hay más de uno se le indica el `@Order`)

```
@Bean
@Order(10)
@GlobalClientInterceptor
ClientInterceptor clientLoggerInterceptor() {
    return new ClientLoggerInterceptor();
}

@Bean
@Order(20)
@GlobalClientInterceptor
ClientInterceptor clientResponseLoggerInterceptor() {
    return new ClientResponseLogger();
}
```

Server Side Interceptor

Para interceptar un mensaje en el servidor se implementa `ServerInterceptor` y se procesa el *request* antes de que se lo envíe al `Servant`.

```
public class ServerRequestLoggerInterceptor implements ServerInterceptor {  
    private static final Logger log = ...  
  
    @Override  
    public <ReqT, RespT> ServerCall.Listener<ReqT> interceptCall(  
        ServerCall<ReqT, RespT> call,  
        Metadata headers, ServerCallHandler<ReqT, RespT> next) {  
        // Here goes your code #1  
        String rpcId = UUID.randomUUID().toString();  
        MethodDescriptor<ReqT, RespT> methodDescriptor = call.getMethodDescriptor();  
        String serviceName = methodDescriptor.getServiceName();  
        String methodName = methodDescriptor.getBareMethodName();  
        log.info("Call {}: {}#{}", rpcId, serviceName, methodName);  
        // Here goes your code #2  
        return next.startCall(call, headers);  
    }  
}
```

```
Call 1a0784cb-d718-4662-966a-e8afeca80ad9:  
trainTicketService.TrainTicketService#GetDestinations
```

Server Side Interceptor

Para interceptar la respuesta, similar al cliente

```
public class GrpcServerResponseInterceptor implements ServerInterceptor {
    private static final Logger log = ...

    @Override
    public <ReqT, RespT> ServerCall.Listener<ReqT> interceptCall(ServerCall<ReqT, RespT> serverCall,
        Metadata metadata, ServerCallHandler<ReqT, RespT> next) {
        return next.startCall(
            new ForwardingServerCall.SimpleForwardingServerCall<>(serverCall) {
                @Override
                public void sendMessage(RespT message) {
                    log.info("Message being sent to client: {}", message);
                    super.sendMessage(message);
                }
            }, metadata);
    }
}
```

Server Side Interceptor

En Spring gRPC para agregar un *interceptor* al servidor se define el @Bean correspondiente como @GlobalClientInterceptor (y si hay más de uno se le indica el @Order)

```
@Bean
@Order(10)
@GlobalServerInterceptor
ServerInterceptor serverRequestLoggerInterceptor() {
    return new ServerRequestLoggerInterceptor();
}

@Bean
@Order(20)
@GlobalServerInterceptor
GrpcServerResponseInterceptor serverResponseInterceptor() {
    return new GrpcServerResponseInterceptor();
}
```


GrpcExceptionHandler

En Spring gRPC se puede implementar la interfaz GrpcExceptionHandler para crear un server interceptor que capture todas las excepciones

```
public interface GrpcExceptionHandler {  
  
    @Nullable  
    StatusException handleException(Throwable exception);  
  
}
```

```
@Bean  
GrpcExceptionHandler grpcExceptionHandler() {  
    return new GlobalGrpcExceptionHandler();  
}
```

GrpcExceptionHandler

```
public class GlobalGrpcExceptionHandler implements GrpcExceptionHandler {
    private final Map<Class<? extends Throwable>, Code> errorCodesByException = Map.of(
        IllegalArgumentException.class, Code.INVALID_ARGUMENT
    );

    @Override
    public @Nullable StatusException handleException(Throwable exception) {
        var resolved = resolveExceptionCode(exception);
        if (resolved == null) {
            return null;
        }
        var mappedException = resolved.exception();
        var message = mappedException.getMessage();
        var rpcStatus = com.google.rpc.Status.newBuilder()
            .setCode(resolved.code().getNumber())
            .setMessage(message != null ? message : "Unknown message")
            .build();
        return StatusProto.toStatusException(rpcStatus);
    }

    ...
}
```

GrpcExceptionHandler

Por ejemplo si en el Servant se arroja una `IllegalArgumentException`

```
@Override
public void getTrainsForDestination(StringValue request, StreamObserver<Train> responseObserver) {
    ...
    if(...) {
        throw new IllegalArgumentException("Invalid destination: %s".formatted(request.getValue()));
    }
    ...
}
```

entonces en el cliente en lugar de recibir UNKNOWN ahora se obtiene:

```
io.grpc.StatusRuntimeException: INVALID_ARGUMENT: Invalid destination: Buenos Aires
    at io.grpc.Status.asRuntimeException(Status.java:532) ~[grpc-api-1.77.1.jar:1.77.1]
    at io.grpc.stub.ClientCalls$BlockingResponseStream.hasNext(ClientCalls.java:731)
    ~[grpc-stub-1.77.1.jar:1.77.1]
```

- Invocaciones a `onError` siguen siendo indispensables si por ejemplo se está dentro de un `StreamObserver` y se necesita finalizar el *stream*



CREDITS

Content of the slides:

- » POD - ITBA

Images:

- » POD - ITBA
- » gRPC Up & Running: Indrasiri, Kuruppu (2020) O'Reilly Media
- » Or obtained from: commons.wikimedia.org

Slides theme credit:

Special thanks to all the people who made and released these awesome resources for free:

- » Presentation template by [SlidesCarnival](https://slidescarnival.com)
 - » Photographs by [Unsplash](https://unsplash.com)
- 